

Deep Learning for Solving Dynamic Models

Session 5: Scaling up, the international real business cycle model

Simon Scheidegger

HEC, University of Lausanne

University of Padova

September 23, 2026

Model and calibration: Brumm & Scheidegger (2017), "Using Adaptive Sparse Grids to Solve High-Dimensional Dynamic Models," *Econometrica* 85(5), 1575–1612.

Method: Azinovic, Gaegauf & Scheidegger (2022), *International Economic Review* 63(4), 1471–1525.

Reference solution: Brumm, Krause, Schaab & Scheidegger (2022), "Sparse Grids for Dynamic Economic Models," *Oxford Research Encyclopedia of Economics and Finance*.

Roadmap of This Lecture

Part A

- ▶ **Part I.** The IRBC model: preferences, production, TFP, adjustment costs, irreversibility, Fischer–Burmeister
- ▶ Equilibrium system + finger exercise

Part B

- ▶ **Part II.** DEQN formulation: states, architecture, loss, Gauss–Hermite, training algorithm
- ▶ **Part III.** Implementation: Keras code, convergence, policy functions, validation
- ▶ Bridge: from solving to estimating (Day 2)

Learning objectives

By the end of this lecture you should be able to:

- ▶ Set up the International Real Business Cycle model with multi-country state space, capital adjustment costs, and irreversibility constraints
- ▶ Derive first-order conditions (Euler equations, consumption sharing) and convert KKT conditions into smooth complementarity functions (Fischer–Burmeister)
- ▶ Formulate the DEQN loss function for multi-country DSGE models and train the network on simulated paths
- ▶ Read the companion notebooks: training loop, policy drift, zero-shock steady state, and the comparison with a sparse-grid time-iteration solution
- ▶ Explain why the state, the network outputs and the quadrature grow linearly in N , and where a grid method explodes instead

Part I

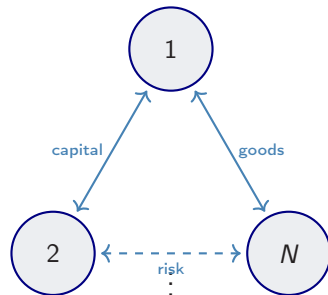
The IRBC Model

Why International Models, and Why IRBC as a Testbed?

The economics. Capital flows, international risk sharing and trade imbalances all need N heterogeneous countries linked through a world capital market. The IRBC model is the workhorse:

- ▶ **Consumption-correlation puzzle** (Backus–Kehoe–Kydlund 1992): complete markets imply $\rho(c^i, c^j) \approx 1$; data say $\rho(c^i, c^j) < \rho(y^i, y^j)$.
- ▶ **Home bias** and the **welfare cost of missing insurance** (Heathcote & Perri 2002).

The computation. State dimension is **linear in N** : $2N$, capital and TFP per country. Smooth but nonlinear; the planner's problem gives a clean target. The benchmark of the JEDC comparison study (Den Haan, Judd & Juillard 2011; Juillard & Villemot 2011; Malin, Krueger & Kübler 2011) and, up to $N = 50$, of Brumm & Scheidegger (2017).



Complete markets

Model Overview

International Real Business Cycle (IRBC) model: Backus, Kehoe & Kydland (1992), in the version of Brumm & Scheidegger (2017), which follows Juillard & Villemot (2011) and adds irreversible investment:

- ▶ N countries, each with **country-specific capital** k^j and **TFP** z^j
- ▶ **Complete markets:** a social planner allocates resources with Pareto weights τ^j
- ▶ **Irreversible investment:** $I^j \geq 0$ (cannot disinvest)
- ▶ **Capital adjustment costs:** $\Gamma^j = \frac{\kappa}{2} k^j \left(\frac{k^{j'}}{k^j} - 1 \right)^2$

Dimensionality

State: $\mathbf{s} = (k^1, \dots, k^N, z^1, \dots, z^N) \in \mathbb{R}^{2N}$

Policy: $(k^{1'}, \dots, k^{N'}, \lambda, \mu^1, \dots, \mu^N) \in \mathbb{R}^{2N+1}$

Preferences

Each country j has **CRRA utility** (with the standard log limit at $\gamma_j = 1$):

$$u^j(c) = \begin{cases} \frac{c^{1-1/\gamma_j} - 1}{1 - 1/\gamma_j}, & \gamma_j \neq 1, \\ \ln c, & \gamma_j = 1. \end{cases}$$

- ▶ γ_j : intertemporal elasticity of substitution for country j
- ▶ **Heterogeneous** across countries: γ_j linearly spaced in $[0.25, 1.0]$ (so risk aversion $1/\gamma_j \in [1, 4]$; the $\ln c$ branch is used at $\gamma_j = 1$)

Notation: in this chapter γ is the **IES**; later chapters on continuous-time HA and climate re-purpose γ for CRRA risk aversion (with ψ then the IES). Restated at the top of each chapter.

Social planner maximizes with Pareto weights τ^j :

$$\max \mathbb{E}_0 \left[\sum_{t=0}^{\infty} \beta^t \sum_{j=1}^N \tau^j u^j(c_t^j) \right]$$

Production

Each country j produces a single good:

$$Y^j = A_{\text{tfp}} \cdot \exp(z^j) \cdot (k^j)^\zeta$$

Calibration of A_{tfp} : chosen so that the **steady state** is consistent with $k_{ss} = 1$:

$$A_{\text{tfp}} = \frac{1 - \beta(1 - \delta)}{\zeta \cdot \beta}$$

Parameter	Value
ζ (capital share)	0.36
β (discount factor)	0.99
δ (depreciation)	0.01

TFP Process

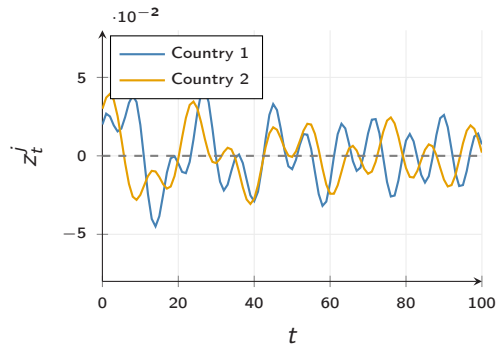
Log TFP follows an **AR(1)** with common and idiosyncratic shocks:

$$z^{j'} = \rho_z z^j + \sigma_e (\varepsilon^j + \varepsilon^{\text{agg}})$$

- ▶ $\varepsilon^j \sim \mathcal{N}(0, 1)$: country-specific
- ▶ $\varepsilon^{\text{agg}} \sim \mathcal{N}(0, 1)$: aggregate shock
- ▶ Total: $N + 1$ independent shocks

$$\rho_z = 0.95, \sigma_e = 0.01$$

σ_e is the *per-component* std., so the per-country innovation std. is $\sigma_e \sqrt{2}$ and the cross-country innovation correlation is $1/2$.



Stylized TFP sample paths

Capital Adjustment Costs

Changing the capital stock is **costly**
($\kappa = 0.5$, the paper's ϕ):

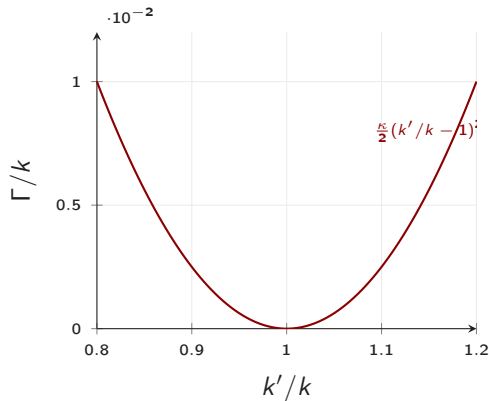
$$\Gamma^j = \frac{\kappa}{2} k^j \left(\frac{k^{j'}}{k^j} - 1 \right)^2$$

Key derivatives:

$$\frac{\partial \Gamma}{\partial k'} = \kappa \left(\frac{k'}{k} - 1 \right)$$

$$\frac{\partial \Gamma}{\partial k} = -\frac{\kappa}{2} \left(\frac{k'}{k} - 1 \right) \left(\frac{k'}{k} + 1 \right)$$

Role: smooth investment, break Tobin's $q=1$, and dampen cross-country **capital flows**.



Adjustment cost per unit of capital

Putting It Together: The Complete Model

Primitives (from the previous slides):

Preferences: $U^j(c) = \frac{c^{1-1/\gamma_j}-1}{1-1/\gamma_j}$

Production: $Y_t^j = A_{\text{tfp}} \exp(z_t^j) (k_t^j)^\zeta$

TFP process: $z_{t+1}^j = \rho_z z_t^j + \sigma_e (\varepsilon_{t+1}^j + \varepsilon_{t+1}^{\text{agg}})$

Adjustment cost: $\Gamma_t^j = \frac{\kappa}{2} k_t^j (k_{t+1}^j/k_t^j - 1)^2$

Constraints:

Aggregate resource (ARC): $\sum_{j=1}^N \left[Y_t^j + (1 - \delta) k_t^j - k_{t+1}^j - \Gamma_t^j - c_t^j \right] = 0$

Irreversibility: $k_t^j \equiv k_{t+1}^j - (1 - \delta) k_t^j \geq 0 \quad \forall j = 1, \dots, N$

The planner chooses $\{c_t^j, k_{t+1}^j\}_{j,t}$ subject to these constraints.

Planner's Problem (Dynamic Optimization)

$$\max_{\{c_t^j, k_{t+1}^j\}} \mathbb{E}_0 \left[\sum_{t=0}^{\infty} \beta^t \sum_{j=1}^N \tau^j \frac{(c_t^j)^{1-1/\gamma_j} - 1}{1 - 1/\gamma_j} \right]$$

subject to:

$$\text{ARC: } \sum_{j=1}^N \left[Y_t^j + (1 - \delta) k_t^j - k_{t+1}^j - \Gamma_t^j - c_t^j \right] = 0$$

$$\text{Irreversibility: } k_{t+1}^j - (1 - \delta) k_t^j \geq 0 \quad \forall j$$

$$\text{Production: } Y_t^j = A_{\text{tfp}} \exp(z_t^j) (k_t^j)^\zeta$$

$$\text{TFP: } z_{t+1}^j = \rho_z z_t^j + \sigma_e (\varepsilon_{t+1}^j + \varepsilon_{t+1}^{\text{agg}})$$

Pareto weights τ^j allocate welfare across countries. We now *attach multipliers* to the constraints to obtain the first-order conditions.

The Lagrangian: Attaching Multipliers

Attach one multiplier per active constraint of the planner's problem:

- ▶ λ_t on the aggregate resource constraint, the **shadow price of world output** at date t .
- ▶ $\mu_t^j \geq 0$ on country j 's irreversibility, the **shadow price of being unable to disinvest**.

$$\mathcal{L} = \mathbb{E}_0 \left[\sum_{t=0}^{\infty} \beta^t \left(\sum_j \tau^j \frac{(c_t^j)^{1-1/\gamma_j}-1}{1-1/\gamma_j} + \lambda_t \sum_j (Y_t^j + (1-\delta)k_t^j - k_{t+1}^j - \Gamma_t^j - c_t^j) + \sum_j \mu_t^j (k_{t+1}^j - (1-\delta)k_t^j) \right) \right]$$

Economic reading:

- ▶ λ_t high $\Rightarrow$ world resources scarce $\Rightarrow$ every country cuts consumption.
- ▶ $\mu_t^j > 0$ only when country j 's irreversibility constraint *binds*.

KKT Conditions for Irreversibility

With the multipliers μ_t^j now defined, optimality of the planner's choice of k_{t+1}^j together with $\mu_t^j \geq 0$ yields the **complementary slackness** structure:

$$\mu^j \geq 0, \quad I^j \geq 0, \quad \mu^j \cdot I^j = 0.$$

Economic reading:

- ▶ $I^j > 0$ (agent invests): constraint slack $\Rightarrow \mu^j = 0$.
- ▶ $I^j = 0$ (constraint binds): $\mu^j \geq 0$ records the shadow value of enforcing $I^j \geq 0$.

The non-negativity $\mu^j \geq 0$ is the *standard KKT sign*: the multiplier on a $\geq$ -constraint is non-negative at the optimum. The smoothed Fischer–Burmeister residual on the next slide encodes this sign restriction *and* the slackness condition $\mu^j I^j = 0$ in one differentiable expression.

Numerical problem: KKT involves min / max, **non-differentiable**, incompatible with SGD. Next slide: smooth it via the Fischer–Burmeister function.

Fischer–Burmeister Complementarity Function

Problem: KKT conditions involve **non-smooth** min/max operators.

Solution: use a smoothed **Fischer–Burmeister** (FB) residual:

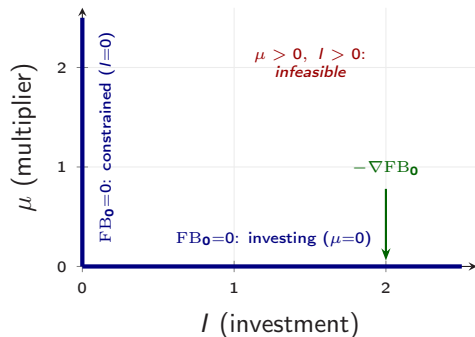
$$\text{FB}_\varepsilon(\mu, I) = \mu + I - \sqrt{\mu^2 + I^2 + \varepsilon^2}$$

$\varepsilon > 0$ small for numerical stability.

$$\text{FB}_0(\mu, I) = 0 \Leftrightarrow \mu \geq 0, I \geq 0, \mu \cdot I = 0.$$

For $\varepsilon > 0$, this exact zero set is slightly smoothed near the origin.

$\Rightarrow$ For $\varepsilon > 0$, **differentiable** everywhere, compatible with SGD.



$\text{FB}_0=0$ exactly on the two blue half-axes (one of μ , I is zero, the other ≥ 0). The open quadrant is infeasible; $-\nabla \text{FB}_0$ (green) pushes a prediction there back to the nearest axis.

FOC: Consumption Sharing

First-order condition w.r.t. c^j :

$$\tau^j \cdot (c^j)^{-1/\gamma_j} = \lambda$$

where λ is the **shadow price** of the aggregate resource constraint.

Solving for consumption:

$$c^j = \left(\frac{\lambda}{\tau^j} \right)^{-\gamma_j}$$

- ▶ λ is the **single** policy variable governing consumption allocation
- ▶ Higher $\lambda \Rightarrow$ resources are scarcer $\Rightarrow$ lower consumption
- ▶ Countries with higher γ_j (lower risk aversion) respond more to λ

FOC: Euler Equation

First-order condition w.r.t. $k^{j'}$:

$$\lambda (1 + \Gamma_{k'}) = \beta \mathbb{E}[\lambda' \cdot \text{MPK}^{j'} - (1 - \delta) \mu^{j'}] + \mu^j$$

where:

- ▶ $\Gamma_{k'} = \kappa \left(\frac{k'}{k} - 1 \right)$ is the marginal adjustment cost
- ▶ $\text{MPK}^j = 1 - \delta + \zeta A_{\text{tfp}} \exp(z^j) (k^j)^{\zeta-1} - \frac{\partial \Gamma}{\partial k}$
- ▶ μ^j is the **irreversibility multiplier**
- ▶ The term $(1 - \delta) \mu^{j'}$ captures the **shadow value** if the constraint binds tomorrow

Euler Equation: Relative Error Form

Rearranging the Euler equation into **relative error** form:

$$\frac{\beta \mathbb{E}[\lambda' \cdot \text{MPK}^{j'} - (1 - \delta) \mu^{j'}] + \mu^j}{\lambda (1 + \Gamma_{k'})} - 1 = 0$$

Why relative error?

- ▶ All N Euler equations are **scale-free**
- ▶ Residuals can be interpreted as **percentage errors** in the optimality condition
- ▶ A residual of 10^{-3} means the agent is off by 0.1%

Brumm & Scheidegger (2017, eq. 43) report this residual without μ^j in the numerator, which is the same thing whenever the constraint is slack today; for the nonsmooth model they combine it with the relative constraint violation (their eq. 46).

Aggregate Resource Constraint

All output must be allocated to consumption, investment, and adjustment costs:

$$\sum_{j=1}^N \left[\underbrace{Y^j + (1 - \delta) k^j}_{\text{resources}} - \underbrace{k^{j'}}_{\text{investment}} - \underbrace{\Gamma^j}_{\text{adj. cost}} - \underbrace{c^j}_{\text{consumption}} \right] = 0$$

- ▶ This is a **single** equation (not N separate ones)
- ▶ Under complete markets, goods flow freely between countries
- ▶ Consumption c^j is determined by λ and the Pareto weights

Complementarity Conditions

For each country j , the Fischer–Burmeister condition enforces irreversibility:

$$\text{FB}_\varepsilon(\mu^j, l^j) = \mu^j + l^j - \sqrt{(\mu^j)^2 + (l^j)^2 + \varepsilon^2} = 0$$

where $l^j = k^{j'} - (1 - \delta) k^j$.

Total system:

Equation	Count	Unknown
Euler equations	N	$k^{1'}, \dots, k^{N'}$
Aggregate resource constraint	1	λ
Fischer–Burmeister	N	$\mu^1, \dots, \mu^N$
Total	$2N + 1$	$2N + 1$

Summary: Equilibrium System

Well-posed system: $2N + 1$ equations in $2N + 1$ unknowns.

N	States	Policies	Equations
2	4	5	5
4	8	9	9
10	20	21	21
50	100	101	101
100	200	201	201

- ▶ Dimensions grow **linearly** in N , no combinatorial explosion
- ▶ Traditional grid methods: $\mathcal{O}(n^{2N})$ grid points
- ▶ DEQNs: a single neural network, trained via SGD

Finger Exercise: IRBC Euler Equations

Exercise

Suppose we **double** the depreciation rate δ from 0.01 to 0.02. The notebook recomputes $A_{\text{tfp}} = (1/\beta - 1 + \delta)/\zeta$ so that $k_{ss} = 1$.

1. Hold A_{tfp} fixed at its baseline value instead. Does steady-state capital rise or fall? Why?
2. In the notebook's normalization k_{ss} stays at 1. What changes instead? Compute the steady-state investment share I_{ss}/Y_{ss} before and after.
3. Confirm your number from the parameter cell's printout of A_{tfp} in `05_01_IRBC_DEQN_smooth.ipynb` (no training needed).

Solution: IRBC Euler Equations

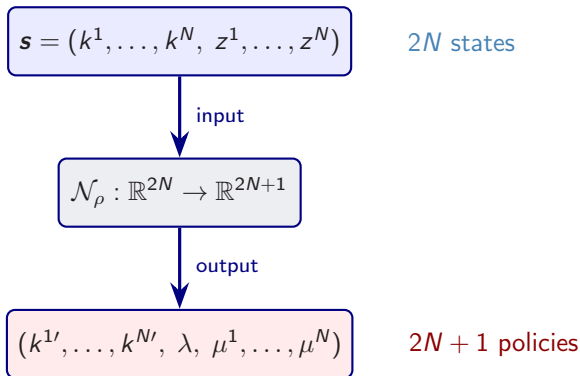
Answer

1. With A_{tfp} fixed, capital **falls**: the steady-state Euler equation is $\zeta A_{\text{tfp}} k^{\zeta-1} = 1/\beta - 1 + \delta$, so a higher δ needs a higher marginal product, and with diminishing returns that means a lower k . Here k_{ss} drops from 1 to $(0.0199/0.0298)^{1/0.64} \approx 0.53$.
2. With k_{ss} pinned at 1, the change lands in A_{tfp} : it rises from 0.0558 to 0.0836. Output rises with it and depreciation doubles, so the investment share $I_{ss}/Y_{ss} = \delta/A_{\text{tfp}}$ **rises** from 0.179 to 0.239; $c_{ss} = A_{\text{tfp}} - \delta$ goes from 0.0458 to 0.0636.
3. The parameter cell prints $A_{\text{tfp}} = 0.083613$ after the change. The trained policy is not needed for any of this: the steady state is pinned by β , δ , ζ alone.

Part II

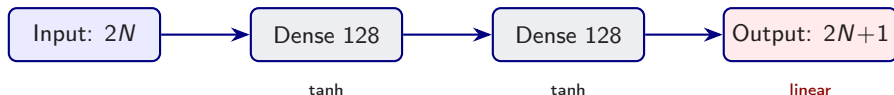
DEQN Formulation for the IRBC

State & Policy Variables



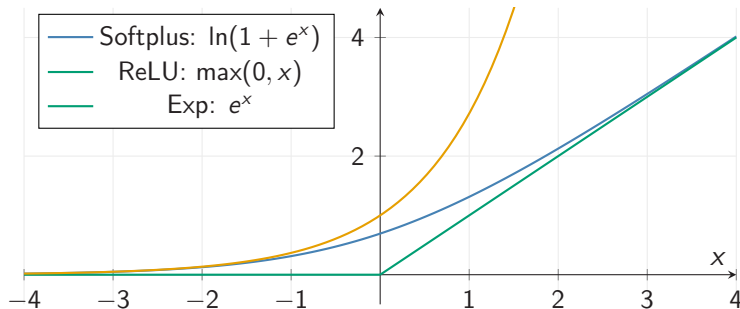
- ▶ The neural network maps the **full state** to **all policy variables simultaneously**
- ▶ For $N = 2$: input dimension 4, output dimension 5
- ▶ Outputs must satisfy economic constraints: a **linear** output layer is mapped through custom per-policy transforms

Neural Network Architecture



- ▶ **Hidden layers:** 2 layers $\times$ 128 units with **tanh** activation
- ▶ **Output layer:** **linear** (zero-initialized); raw outputs pass through custom transforms
- ▶ Transforms enforce the constraints: $k^{j'} = k^j \exp\{\bar{g} \tanh r\}$ in 05_01, $k^{j'} = (1 - \delta)k^j + k^j \bar{t} \sigma(r)$ with $I^j \geq 0$ by construction in 05_02; $\lambda > 0$ via $\exp \cdot \tanh$; $\mu^j \geq 0$ via softplus (05_02 only)
- ▶ Total parameters: $(2N \cdot 128 + 128) + (128 \cdot 128 + 128) + (128 \cdot (2N + 1) + 2N + 1)$

Enforcing Positive, Feasible Outputs



The notebooks keep a **linear** output and apply bounded transforms: $k^{j'} = k^j \exp\{\bar{g} \tanh r\}$ and $\lambda = \lambda_{\text{ref}} \exp\{s \tanh r\}$, the $\tanh$ caps the exponent, so no overflow (unlike a raw e^x), while the inequality multipliers μ^j use **softplus**.

DEQN Loss Function for the IRBC

The loss is the **sum of squared residuals** across all equilibrium conditions; $\ell_\rho = 0$ iff every condition holds exactly.

Smooth benchmark (notebook 05_01):

$$\ell_\rho^{\text{smooth}} = \frac{1}{B} \sum_{i=1}^B \left[\sum_{j=1}^N (\text{Euler}_j^{(i)})^2 + (\text{ARC}^{(i)})^2 \right]$$

Irreversibility extension (notebook 05_02): add the Fischer–Burmeister block,

$$\ell_\rho^{\text{irrev}} = \ell_\rho^{\text{smooth}} + \frac{1}{B} \sum_{i=1}^B \sum_{j=1}^N (\text{FB}_j^{(i)})^2.$$

with B the batch size:

- ▶ $\text{Euler}_j = \frac{\beta \mathbb{E}[\lambda' \text{MPK}' - (1 - \delta)\mu'] + \mu}{\lambda(1 + \Gamma_{k'})} - 1$ (smooth: $\mu \equiv 0$)
- ▶ $\text{ARC} = \sum_j [Y^j + (1 - \delta)k^j - k^{j'} - \Gamma^j - c^j] / \sum_j [Y^j + (1 - \delta)k^j]$ (unit-free, as the paper's eq. 44)
- ▶ $\text{FB}_j = \text{FB}_\epsilon(\mu^j/\lambda, l^j/k^j)$ on the unit-free pair, same zero set (05_02 only)

Gauss–Hermite Quadrature

The Euler equation requires computing $\mathbb{E}[\cdot]$ over $N + 1$ normal shocks.

Gauss–Hermite quadrature in 1D:

$$\mathbb{E}[f(\varepsilon)] = \int f(\varepsilon) \phi(\varepsilon) d\varepsilon \approx \sum_{q=1}^Q w_q f(\varepsilon_q)$$

Tensor product for $N + 1$ dimensions:

$$\mathbb{E}\left[f(\varepsilon^1, \dots, \varepsilon^N, \varepsilon^{\text{agg}})\right] \approx \sum_{q_1=1}^Q \cdots \sum_{q_{N+1}=1}^Q \left(\prod_d w_{q_d} \right) f(\varepsilon_{q_1}, \dots, \varepsilon_{q_{N+1}})$$

N	Shocks	Q	Quad. nodes
2	3	3	27
4	5	3	243
10	11	3	$\sim 177\text{K}$

What the companion notebooks do. Both 05_01 and 05_02 use the monomial rule of Brumm & Scheidegger (2017, Appendix C; Judd 1998, §7.5): $2(N + 1)$ nodes at $\pm\sqrt{N + 1}$ on each shock axis with equal weights, exact for polynomials of degree 3, so 6 nodes for $N = 2$. The tensor-product rule above is shown for contrast.

Computing the Euler Expectation

Step-by-step for each quadrature node q :

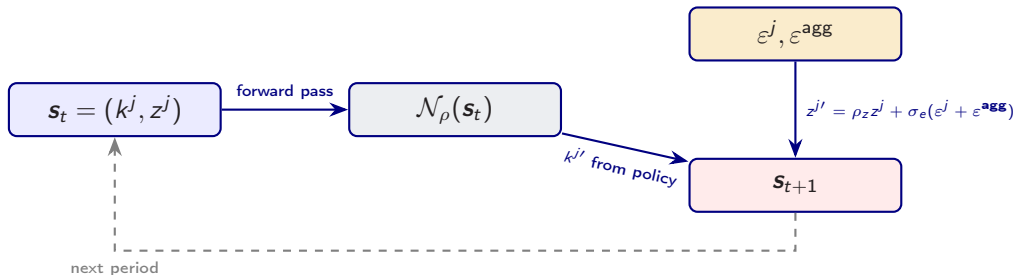
1. **Current state** $\mathbf{s}_t = (k^1, \dots, k^N, z^1, \dots, z^N)$
2. **Current policy** $\mathcal{N}_\rho(\mathbf{s}_t) \rightarrow (k^{1'}, \dots, k^{N'}, \lambda)$ (smooth, $N + 1$ outputs; the irreversible extension appends $\mu^1, \dots, \mu^N$)
3. **Next-period state** at node q :

$$k_q^{j'} = k^{j'}, \quad z_q^{j'} = \rho_z z^j + \sigma_\epsilon (\epsilon_q^j + \epsilon_q^{\text{agg}})$$

4. **Next-period policy**: evaluate $\mathcal{N}_\rho(\mathbf{s}_{t+1,q})$
5. **Accumulate**: E-term $= \sum_q w_q [\lambda'_q \cdot \text{MPK}_q^j - (1 - \delta) \mu_q^{j'}]$

Each quadrature node requires a **forward pass** through the network, but all nodes in a batch can be evaluated in parallel.

State Transition



- **Capital:** $k^{j'}$ is a policy output (endogenous)
- **TFP:** $z^{j'}$ evolves exogenously via AR(1) + shocks
- For simulation: draw $\varepsilon \sim \mathcal{N}(0, 1)$
- For expectations: use quadrature nodes ε_q

Training Algorithm

Algorithm 1: DEQN Training for IRBC (persistent simulation)

Input: Network $\mathcal{N}_\rho$, learning rate η , segments S , monomial nodes/weights
Initialize M trajectory heads $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(M)}$ from a box of feasible states
for segment $s = 1, \dots, S$ **do**
 Simulate: for $t = 1, \dots, T$, sample shocks and step each head one period under $\mathcal{N}_\rho$; a head that leaves the admissible box is replaced and counted (repairs)
 Flatten the $M \cdot T$ recorded states into a training batch
 for pass $p = 1, \dots, P$ **over mini-batches of the segment do**
 Compute loss ℓ_ρ on the segment (smooth: Euler + ARC; irreversible: also FB)
 Update: $\rho \leftarrow \rho - \eta \nabla_\rho \ell_\rho$ (Adam; η stepped down twice over the run)
 end for
 Continue: $\mathbf{X}^{(m)} \leftarrow$ terminal state of trajectory m (*no reset to steady state*)
 Monitor: policy drift on $\mathbf{X}_{\text{anchor}}$; flag stable when below tolerance
end for
Diagnostic: zero-shock SSS check from dispersed feasible starts
Output: Trained $\mathcal{N}_{\rho^*}$

Scalability in N

N	States	Policies	NN params	Grid pts ($n=10$)
2	4	5	17.8K	10^4
4	8	9	18.8K	10^8
10	20	21	21.9K	10^{20}
50	100	101	42.5K	10^{100}
100	200	201	68.1K	10^{200}

- ▶ NN parameters grow **linearly** in N (two 128-unit layers; only the input and output layers scale); grid points grow as n^{2N}
- ▶ DEQN cost per update: $\mathcal{O}(|\rho| B (2N + 3))$ with the monomial rule, against $\mathcal{O}(|\rho| B Q^{N+1})$ with tensor-product Gauss–Hermite; at large N , quasi-Monte Carlo or sparse-grid quadrature
- ▶ Theoretical underpinning: **deep ReLU networks** beat the curse of dimensionality for sparse-grid-representable functions (Montanelli & Du 2019)

Recap: Three Building Blocks

Model

N -country IRBC with irreversible investment, adjustment costs, complete markets. $2N + 1$ equilibrium equations.

Network

$\mathcal{N}_\rho : \mathbb{R}^{2N} \rightarrow \mathbb{R}^{2N+1}$ with tanh hidden layers, linear output mapped through bounded transforms. All policies in one forward pass.

Training

Persistent simulation: one continuing trajectory ensemble, Adam on the sum of squared residuals.

Relative to Brock–Mirman: one country $\rightarrow N$; state $(K, z) \rightarrow (k^{1:N}, z^{1:N})$; one Euler equation $\rightarrow N$ Euler + 1 ARC + N Fischer–Burmeister; a closed form (at $\delta = 1$) $\rightarrow$ none. The IRBC is the natural **scaling test** for the method.

$\Rightarrow$ **Next:** implementation in TensorFlow/Keras and results.

Part III

Implementation & Results

Notebook Overview

Three notebooks accompany this lecture, all in `day1/code/05_irbc/`:

1. `05_01_IRBC_DEQN_smooth.ipynb`, *smooth benchmark, in-class walk-through*
 - ▶ Parameters, monomial quadrature, Keras network, `compute_residuals` (Euler + ARC), persistent-simulation training loop
 - ▶ Policy drift, zero-shock steady state, and the comparison with the sparse-grid time-iteration solution of Brumm & Scheidegger (2017)
2. `05_02_IRBC_DEQN_irreversible.ipynb`, *irreversible-investment extension*
 - ▶ KKT multipliers μ^j , feasibility-by-construction policy, Fischer–Burmeister complementarity loss
 - ▶ Same training loop and diagnostics; where the constraint binds, checked against the sparse-grid solution
3. `05_03_IRBC_Exercise.ipynb`, *hands-on exercise*
 - ▶ Task 1: steady-state comparative statics in the lecture's calibration, A_{tfp} fixed against A_{tfp} renormalized
 - ▶ Task 2: inverse-loss weighting (Session 3) on the residual magnitudes from 05_02's training log
 - ▶ Task 3: a two-minute smoke run of 05_01 with a re-weighted loss

Parameters

Symbol	Name	Value	Description
β	Discount factor	0.99	Quarterly calibration
ζ	Capital share	0.36	Cobb–Douglas
δ	Depreciation	0.01	Low depreciation
ρ_z	TFP persistence	0.95	Highly persistent
σ_e	Shock std. dev.	0.01	Small shocks
κ	Adjustment cost	0.50	Moderate costs
$\gamma_{\min}$	Min IES	0.25	High risk aversion
$\gamma_{\max}$	Max IES	1.00	Log utility
k_{ss}	Steady-state capital	1.00	Normalization

Pareto weights: $\tau^j = c_{ss}^{1/\gamma_j}$ with $c_{ss} = A_{\text{tfp}} - \delta$, so that $\lambda_{ss} = \tau^j c_{ss}^{-1/\gamma_j} = 1$ for every j despite heterogeneous γ_j . Table II of the paper has $\tau^j = A_{\text{tfp}}^{1/\gamma_j}$, inherited from Juillard & Villemot (2011), where $\delta = 0$ and hence $c_{ss} = A_{\text{tfp}}$.

Steady State

At steady state ($z^j = 0$, $k^j = k_{ss} = 1$, $\mu^j = 0$):

$$A_{\text{tfp}} = \frac{1 - \beta(1 - \delta)}{\zeta\beta} = \frac{1 - 0.99 \cdot 0.99}{0.36 \cdot 0.99} \approx 0.0559$$

$$Y_{ss} = A_{\text{tfp}} \cdot k_{ss}^{\zeta} = A_{\text{tfp}} \approx 0.0559$$

$$I_{ss} = \delta \cdot k_{ss} = 0.01$$

$$c_{ss} = Y_{ss} - I_{ss} \approx 0.0459$$

Verification: the ARC should be satisfied: $\sum_j [Y_{ss} - I_{ss} - c_{ss}] = 0 \checkmark$

The Euler equation at steady state gives the MPK = $1/\beta$, which pins down A_{tfp} .

Fischer–Burmeister Function

```
FB_EPS = 1e-4
def fischer_burmeister(a, b):
    return a + b - tf.sqrt(a * a + b * b + FB_EPS * FB_EPS)
# applied to the unit-free pair (mu / lambda, I / k)
```

Investment:

```
def investment(k, kp):
    return kp - (1.0 - delta) * k
```

The FB function is:

- ▶ **Differentiable** everywhere for $\varepsilon > 0$ (unlike $\min(\mu, I)$)
- ▶ **Exact zero set** at $\varepsilon = 0$: $\mu \geq 0, I \geq 0, \mu \cdot I = 0$
- ▶ The small ε keeps the gradient finite at the origin, where $\sqrt{\mu^2 + I^2}$ is not differentiable

Cost Function Structure

```
def compute_residuals(states, model):
    kp, lamb, mu = policy(states, model)
    # For each monomial node q:
    #     next state (kp, z'_q), evaluate policy there,
    #     accumulate the Euler integrand
    # Euler residual (relative error), N columns
    # ARC residual, one column, unit-free
    # FB residual on (mu/lambda, I/k), 05_02 only
    loss = (mean(euler**2) + mean(arc**2)
            + mean(fb**2))
    return loss, euler, arc, fb, kp, lamb, mu
```

Key point: the whole residual is a TensorFlow graph, so GradientTape differentiates the loss through the policy at today's state and at every next-period node.

Persistent-Simulation Training

The companion notebooks train on **one continuing trajectory ensemble**, with no resets to the steady state.

- ▶ Maintain $M = \text{N_TRAJECTORIES}$ trajectory heads $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(M)}$.
- ▶ Each **segment**: simulate the heads for $T = \text{SIMULATION_LENGTH}$ periods under the *current* policy; flatten into $M \cdot T$ training states; run a fixed number of mini-batch passes.
- ▶ Continue from the terminal states: $\mathbf{X}_{\text{start}} \leftarrow \mathbf{X}_{\text{end}}$. The ensemble **co-evolves** with the policy.
- ▶ `SAMPLING_MODE = "exogenous"` draws from a box instead. Here that fails: the network finds a policy with small residuals on the box whose capital explodes under simulation. The Euler equations alone admit non-transversal paths; the ergodic set rules them out.

Why this works without a separate burn-in. The policy is feasible by construction:

- ▶ smooth (05_01): $k^{j'} = k^j \exp\{\bar{g} \tanh r_j(\mathbf{s})\}$, positive, growth capped at $\bar{g} = 0.1$ per quarter;
- ▶ irreversible (05_02): $k^{j'} = (1 - \delta)k^j + \max\{0, \tilde{k}^{j'} - (1 - \delta)k^j\}$, smoothed, with $\tilde{k}^{j'}$ the smooth head;
- ▶ a head that still leaves the admissible box is replaced and counted (repairs, zero in every stored run).

Diagnostics: Time-Invariance and Zero-Shock SSS

Time invariance. The architecture has no calendar-time input, so any fixed weight vector defines a stationary recursive policy. The *numerical* question is whether SGD has stopped moving the policy function.

- ▶ Evaluate the policy on a fixed set of holdout states X_{anchor} after each monitoring interval.
- ▶ Report `policy_drift_rms` and `policy_drift_max`.
- ▶ When both fall below `TIME_INVARIANCE_TOL_RMS / _MAX`, the run is flagged **stable**.

Zero-shock stochastic steady state (SSS). Iterate the learned policy from `ZERO_SHOCK_N_STARTS` dispersed feasible starts with $\varepsilon_t^j = \varepsilon_t^{\text{agg}} = 0$.

- ▶ All paths should converge to a common point.
- ▶ In a sensible SSS: $I^j \approx \delta k^j$ (investment replaces depreciation) and, in 05_02, the irreversibility multiplier $\mu^j \approx 0$.

Persistent-Simulation Training: What to Monitor

Four signals carry the convergence story:

1. **Total loss** falls across training segments (the *loss-vs-segment* panel in the notebooks), with a visible drop at each learning-rate step.
2. **Per-residual mean magnitudes**: `mean_abs_euler`, `mean_abs_arc`, and (in 05_02) `mean_abs_fb`.
3. **Policy drift on the holdout states** crosses from **moving** to **stable**: only then has the recursive policy stopped changing.
4. **Zero-shock SSS** converges from dispersed feasible starts to a common point with $l^j \rightarrow \delta k^j$ and (in 05_02) $\mu^j \rightarrow 0$.

A fifth, external check closes both notebooks: the trained policy is evaluated at the same states as the sparse-grid time-iteration solution of Brumm & Scheidegger (2017), and the two are compared directly.

Convergence: Measured Residuals

Stored teaching runs, laptop CPU. Residuals evaluated on states the trained policy visits (ergodic set) and on the paper's box $[0.8, 1.2]^2 \times [-0.16, 0.16]^2$.

Notebook	States	mean Euler	max Euler	mean ARC	$ k'_{\text{DEQN}} - k'_{\text{SG}} /k'$
05_01 smooth	ergodic set	2.1×10^{-5}	3.5×10^{-4}	4.7×10^{-5}	8.5×10^{-5}
05_01 smooth	paper box	2.7×10^{-4}	3.0×10^{-3}	9.6×10^{-4}	1.1×10^{-3}
05_02 irreversible	ergodic set	2.0×10^{-5}	3.6×10^{-4}	4.6×10^{-5}	5.4×10^{-4}
05_02 irreversible	paper box	2.4×10^{-3}	1.8×10^{-2}	2.1×10^{-3}	2.9×10^{-3}
05_01, $N = 10$	ergodic set	8.3×10^{-5}	5.8×10^{-4}	3.4×10^{-5}	

- ▶ All residuals are **relative errors**: 10^{-4} is a 0.01% wedge in the optimality condition.
- ▶ The last column compares the DEQN's $k^{j'}$ with the adaptive-sparse-grid time-iteration solution at the same states; the sparse grid's own Euler error under the notebooks' quadrature rule is 8.4×10^{-5} on the box and 2.4×10^{-5} on its path, so on the ergodic set the two methods are equally accurate, and off it the sparse grid, which was solved on the whole box, is the better one.
- ▶ Every number is printed by a stored notebook cell.

What the Notebooks Plot

Both notebooks

- ▶ **Capital policy** $k^{j'}$ against k^j on the simulated states: just off the 45-degree line, slope below one; the crossing is the stochastic steady state.
- ▶ **The simulated** (k^1, k^2) **states**: an ellipse along the diagonal. The common shock moves both stocks together, the idiosyncratic shocks spread them.
- ▶ **Loss, residuals and policy drift** against the training segment.

05_02 only

- ▶ **Complementarity scatter** l^j/k^j against μ^j/λ : every point on one of the two axes.
- ▶ At this calibration ($\delta = \sigma = 1\%$ a quarter) the constraint never binds on the simulated path, in either method. On uniform draws from the paper's box the sparse grid binds on 25% of (state, country) pairs; the DEQN, trained on simulated states only, binds on 23% and agrees with it for 97% of pairs: the smoothed maximum in its head carries the kink, though its multipliers there are untrained.
- ▶ Hence Brumm & Scheidegger (2017) evaluate the nonsmooth model on uniform draws over the box.

Economic Diagnostics

Consumption sharing:

- ▶ Under complete markets, c^1 and c^2 should be **positively correlated**
- ▶ Perfect risk sharing: c^1/c^2 is constant
- ▶ With heterogeneous γ : ratio varies with λ

Reconnects with the **consumption-correlation puzzle** from the opening slide of Part I: complete markets predict $\rho(c^1, c^2) \approx 1$, but in data $\rho(c^1, c^2) < \rho(y^1, y^2)$; closing the gap needs incomplete markets or frictions.

Trade balance (one line to add if you want it):

- ▶ $TB^j = Y^j - c^j - I^j - \Gamma^j (> 0 \text{ when } j \text{ exports})$
- ▶ Sums to zero across countries by the ARC

Ergodic distribution:

- ▶ The (k^1, k^2) scatter over the simulated states
- ▶ Concentrated around $(k_{ss}, k_{ss}) = (1, 1)$; each k^j stays within $[0.83, 1.23]$ over 8,192 simulated states
- ▶ Elongated along the diagonal: the common shock dominates

Complementarity check (05_02):

- ▶ Scatter of μ^j/λ against I^j/k^j
- ▶ Points on the axes: $\mu = 0$ or $I = 0$
- ▶ $\mu \cdot I$ is the headline number: where the constraint is slack, $FB_\epsilon \approx \mu/\lambda$, which sits at the floor of the softplus head

Validation: Euler LHS vs. RHS

Key diagnostic: compare both sides of the Euler equation on a test set.

LHS: $\lambda \cdot (1 + \Gamma_{k'})$

RHS: $\beta \mathbb{E}[\lambda' \cdot \text{MPK}' - (1 - \delta)\mu'] + \mu$

Targets:

- ▶ Points should lie on the **45-degree line**
- ▶ Mean relative error $< 10^{-3}$
- ▶ Max relative error $< 10^{-2}$

Also check:

- ▶ ARC residual ≈ 0 across all states
- ▶ $\mu^j \approx 0$ whenever $I^j > 0$
(complementarity)
- ▶ $I^j \geq 0$ everywhere (irreversibility)

These diagnostics confirm the network has learned a valid **approximate equilibrium**.

Bridge: from solving to estimating

What we leave with today. An IRBC DEQN trained with a single persistent-simulation pipeline and equal loss weights, checked against a sparse-grid time-iteration solution. The same code with $N = 10$ (20 states, 21 policies, 22 quadrature nodes) trains in eight minutes on a laptop CPU to a mean Euler error of 8×10^{-5} . NAS-tuned width and ReLoBRaLo-balanced loss components, from Session 3 this afternoon, are the refinements to reach for when the equal-weight recipe stalls.

Day 1, in one line.

- ▶ The equilibrium residual *is* the loss (Session 2).
- ▶ Architecture and loss weights are the two knobs that decide whether training works (Session 3).
- ▶ Cohorts and constraints do not change the principle (Session 4); neither does a $2N$ -dimensional state (today).

Tomorrow.

- ▶ Solving is not the end: a trained solver is expensive to re-run. Deep surrogates and Gaussian processes *amortize* it (Session 6).
- ▶ That makes structural estimation tractable (Session 7).
- ▶ Then the same residual idea in continuous time: PDEs, HJB, and the coupled HJB–KFE system (Sessions 8–9).

Questions?

Simon Scheidegger

HEC, University of Lausanne

<https://sischei.github.io>

Materials: https://github.com/sischei/deep_learning_padova_0926

Next up:

AutoDiff → Sequence Space →
OLG → Heterogeneous Agents
(Young's method)